



Définition d'une primitive

La relation fondamentale

Soit f une fonction définie sur un intervalle I .

On dit que la fonction F est **une primitive** de f sur I si :

1. F est dérivable sur I .
2. Pour tout $x \in I$, on a : $F'(x) = f(x)$

Unicité à une constante près

Toutes les primitives d'une même fonction diffèrent d'une **constante**.

Si F est une primitive de f , alors toute primitive G de f s'écrit :

$$G(x) = F(x) + C \quad (\text{avec } C \in \mathbb{R})$$

Primitives d'un polynôme

Fonctions polynomiales (Degré 1 à 3)

Soient a, b, c et d des nombres réels.

1. Fonction affine (Degré 1) :

Si $f(x) = ax + b \implies F(x) = \frac{1}{2}ax^2 + bx$

2. Fonction du second degré (Degré 2) :

Si $f(x) = ax^2 + bx + c \implies F(x) = \frac{1}{3}ax^3 + \frac{1}{2}bx^2 + cx$

3. Fonction du troisième degré (Degré 3) :

Si $f(x) = ax^3 + bx^2 + cx + d \implies F(x) = \frac{1}{4}ax^4 + \frac{1}{3}bx^3 + \frac{1}{2}cx^2 + dx$

Règle générale

Pour tout entier $n \geq 0$, une primitive de la fonction $x \mapsto x^n$ sur $\mathbb{R}$ est la fonction $x \mapsto \frac{1}{n+1}x^{n+1}$.

Primitives trigonométriques

Fonctions sinusoidales

Très utiles en physique (signaux périodiques, cinématique), avec ω la pulsation et φ la phase à l'origine.

Pour la fonction :

$$f(t) = A \cos(\omega t + \varphi)$$

Les primitives sont :

$$F(t) = \frac{A}{\omega} \sin(\omega t + \varphi) + C$$

Pour la fonction :

$$f(t) = A \sin(\omega t + \varphi)$$

Les primitives sont :

$$F(t) = -\frac{A}{\omega} \cos(\omega t + \varphi) + C$$

Méthode d'Euler

Le problème de Cauchy

On cherche à construire une approximation de la courbe d'une primitive $y(t)$ lorsque l'on connaît :

1. Son équation différentielle : $y'(t) = f(t, y)$
2. Sa condition initiale (point de départ) : $y(t_0) = y_0$

L'algorithme de construction

On choisit un **pas** d'avancement petit, noté h .

Sachant que la tangente donne une bonne approximation locale, on avance de proche en proche grâce à la formule :

$$y_{n+1} = y_n + h \times f(t_n, y_n)$$

Et le temps avance d'un pas h : $t_{n+1} = t_n + h$.

Au-delà d'Euler

Remarque : Hors programme 1re STI2D

La méthode d'Euler utilise la pente **uniquement au début** de l'intervalle $[t_n, t_{n+1}]$. Si la courbe tourne rapidement, l'erreur s'accumule vite (méthode d'ordre 1).

En ingénierie, on utilise les **méthodes de Runge-Kutta (RK)** pour une meilleure précision.

Le principe de Runge-Kutta

Pour résoudre $y' = f(t, y)$, les méthodes de Runge-Kutta évaluent la dérivée (la pente) en **plusieurs points** intermédiaires entre t_n et t_{n+1} .

Elles font ensuite une **moyenne pondérée** de ces pentes pour calculer une trajectoire beaucoup plus précise sans avoir besoin de réduire le pas h à l'excès.

Méthodes RK2 et RK4

Méthode RK2 (Point milieu)

Plutôt qu'une seule pente, on en calcule deux :

$$k_1 = f(t_n, y_n) \quad (\text{Pente initiale})$$

$$k_2 = f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2}k_1\right) \quad (\text{Pente au milieu estimée})$$

$$y_{n+1} = y_n + h \times k_2$$

La célèbre RK4 (Le standard de l'ingénierie)

Utilise **4 estimations** de pentes successives :

$$k_1 = f(t_n, y_n)$$

$$k_2 = f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2}k_1\right)$$

$$k_3 = f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2}k_2\right)$$

$$k_4 = f(t_n + h, y_n + hk_3)$$

Moyenne pondérée finale :

$$y_{n+1} = y_n + \frac{h}{6} (k_1 + 2k_2 + 2k_3 + k_4)$$

Programmation Python

1. Méthode d'Euler (Boucle WHILE)

```
def euler(f, y0, t0, t_cible, h):  
    y = y0  
    t = t0  
    while t < t_cible:  
        y = y + h * f(t, y)  
        t = t + h  
    return y
```

2. Méthode RK4 (Boucle FOR)

```
def rk4(f, y0, t0, h, N):  
    y = y0  
    t = t0  
    for i in range(N):  
        k1 = f(t, y)  
        k2 = f(t + h/2, y + (h/2)*k1)  
        k3 = f(t + h/2, y + (h/2)*k2)  
        k4 = f(t + h, y + h*k3)  
        y = y + (h/6) * (k1 + 2*k2 + 2*k3 + k4)  
        t = t + h  
    return y
```